

Practica 8 - Estructuras de control y Sentencias

Objetivo: reconocer las diferencias entre las implementaciones de las estructuras de control de los distintos lenguajes.

Ejercicio 1: Una sentencia puede ser simple o compuesta, ¿Cuál es la diferencia?

- Una sentencia simple: Es una unica instruccion que se ejecuta en el programa.

```
int x = 5 + 10;
```

- Una sentencia compuesta:
 - Se utilizan para ejecutar una serie de sentencias como si fueran una sola.
 - Los delimitadores varian segun el lenguaje, por ejemplo en C, C++, Java, se usan las llaves ' [] ' mientras que en pascal se usa Begin y End.

Ejercicio 2: Analice como C implementa la asignación.

Una restriccion de como se implementa la asignacion en C, es que, del lado izquierdo de la asignacion, debe tener si o si un L-Valor (una variable, una posicion de memoria), no puede tener un R-Valor (valor,resultado de una expresion).

En C, la asignacion se implementa teniendo en cuenta las siguientes características:

- Operador de asignacion: Se utiliza el sino igual ' = ' para la asignacion.
- L-Valor y R-Valor: En general asigna el valor de la expresion de la derecha (R-Valor), a la variable que se encuentra a la izquierda de la asignacion (L-Valor).
- Asignacion como expresion: en C, la asignacion es en si misma una expresion que devuelve el valor asignado. Permite realizar asignaciones multiples ej (a = b = c = d = 0;) y usar la asignacion dentro de otras expresiones (ej if (i=30))

Ejercicio 3: ¿Una expresión de asignación puede producir efectos laterales que afecten al resultado final, dependiendo de cómo se evalúe? De ejemplos.

Si, una expresion de asignacion en C, puede producir efectos secundarios que influyen en el resultado final de un programa, dependiendo del contexto en que se evalúe.

```
a = b = c = 0;
```

En este ejemplo que esta en la teoria se ve como el efecto secundario/lateral de cada asignacion afecta a la siguiente asignacion (hay que tener en cuenta que en C , la asignacion se evalua de derecha a izquierda)

Ejercicio 4: Qué significa que un lenguaje utilice circuito corto o circuito largo para la evaluación de una expresión. De un ejemplo en el cual por un circuito de error y por el otro no.

- **Circuito Corto (Evaluación Perezosa):** La evaluación de una expresión lógica (con AND u OR) se detiene tan pronto como se conoce el resultado. Los operandos se evalúan de izquierda a derecha, y si el resultado se puede determinar con los primeros operandos, los restantes no se evalúan.
- **Circuito Largo:** Todos los operandos de la expresión lógica se evalúan, incluso si el resultado ya se conoce.
- **Conclusion:** Circuito corto lo que hace es determinar lo antes posible el resultado de la condicion, dado que si se tienen varias condiciones, y al menos una es falsa, ya se sabe que toda la condicion completa va a ser falsa, corta la evaluacion lo antes posible, entonces se evita preguntar por las otras condiciones siguientes.

Ejemplo:

```
int x = 0;

if (x != 0 && 10 / x > 1) {
    printf("Resultado: %d\n", 10 / x);
} else {
    printf("No se pudo calcular\n");
}
```

- **Circuito Corto:** Si `x` es 0, la primera parte de la condición (`x != 0`) es falsa. Dado que la condición completa es un AND , el resultado es falso sin importar el valor de la segunda parte. Por lo tanto, `10 / x` **no se evalúa**, evitando un error de división por cero.
- **Circuito Largo:** Si C usara circuito largo, ambos lados del AND se evaluarían. Incluso si `x != 0` fuera falso, se intentaría calcular `10 / x`, lo que causaría un error por intentar dividir por 0, y se terminaria el programa.

Ejercicio 5: ¿Qué regla define Delphi, Ada y C para la asociación del else con el if correspondiente? ¿Cómo lo maneja Python?

- **Delphi, Ada, C:** Utilizan la regla de que el `else` se asocia con el `if` no emparejado más cercano (más interno) dentro del mismo bloque de código.

- **Python:** No usa llaves ni palabras clave como `begin / end` para definir bloques. En cambio, usa la indentación (sangría) para determinar la estructura del código. El `else` se asocia con el `if` que tiene la misma indentación. Esto elimina la ambigüedad del "dangling else".

Ejercicio 6: ¿Cuál es la construcción para expresar múltiples selección que implementa C? ¿Trabaja de la misma manera que la de Pascal, ADA o Python?

- **C:** Implementa la selección múltiple con la sentencia `switch-case`.
- **Diferencias:**
 - **C vs. Pascal:** C no tiene el problema de "no acción" de Pascal, pero requiere `break` para evitar el "falling through" de un `case` al siguiente. Pascal no tiene "falling through" por defecto.
 - **C vs. Ada:** Ada es más estricto que C. En Ada, el `case` debe cubrir todos los valores posibles, o se debe usar `others`. Ada no tiene "falling through".
 - **C vs. Python:** Python no tiene una estructura `switch-case` como C. En su lugar, usa `if-elif-else` para la selección múltiple, y la indentación define los bloques.

Ejercicio 7: Sea el siguiente código:

<pre>..... var i, z:integer; Procedure A; begin i:= i +1; end; begin z:=5;</pre>	<pre>for i:=1..5 do begin z:=z*5; A; z:=z + i; end; end;</pre>
--	--

a- Analice en las versiones estándar de ADA y Pascal, si este código puede llegar a traer problemas. Justifique la respuesta.

- **Pascal::**
 - Según la teoría, el estándar de Pascal tiene restricciones sobre la modificación de la variable iteradora 'i', que se usa en el `for`.
 - El estándar de Pascal establece que la variable iteradora no puede ser modificada por ejemplo en este caso dentro del bucle, cuando se llama al procedimiento A, se intenta incrementar el valor de i, esto puede llevar a un comportamiento indefinido.
 - Otra cosa a tener en cuenta es que el valor de la variable 'i', ni bien termina el `for`, se considera con valor indefinido en Pascal estándar. Esto significa que no se puede predecir con seguridad qué valor tendrá 'i' después de la ejecución del bucle.
- **Ada:**

- Ada también tiene sus propias reglas para los bucles `for`.
- En Ada, la variable de control del bucle (`i` en este caso) se declara implícitamente dentro del bucle y su alcance se limita al bucle.
- No se permite la modificación explícita de la variable de control dentro del bucle.
- En el código que nos dan, `i` se declara fuera del bucle, por lo que la modificación de `i` en el procedimiento `A` se refiere a la variable global, lo cual está permitido, pero generalmente se considera una mala práctica por razones de claridad y mantenimiento del código.
- En Ada, el código no generaría un error en tiempo de compilación por la modificación de la variable `i`, pero sí podría ser problemático en términos de legibilidad y mantenimiento.

b- Comente qué sucedería con las versiones de Pascal y ADA, que Ud. utilizó.

- **Pascal:**

- Has observado que en la versión de Pascal que utilizaste, `i` conserva el valor 5 después del bucle.
- Esto **no coincide con el estándar de Pascal**, que establece que el valor de la variable de control después del bucle es indefinido.
 - Las diferentes versiones de los compiladores de Pascal pueden tener extensiones o comportamientos no estándar.

- **Ada:**

- Ada impone reglas estrictas sobre los bucles `for`, y la modificación de la variable de control del bucle está prohibida dentro del bucle.
- La versión de Ada permite modificar una variable global, por lo que es importante ser consciente de que esto puede hacer que el código sea más difícil de entender y mantener.

Ejercicio 8: Sea el siguiente código escrito en Pascal

```
var puntos: integer;
begin
...
case puntos of
  1..5: write('No puede continuar');
  10: write('Trabajo terminado')
end;
..
```

Sintaxis trasladada a ADA:

La declaración para hacer un `case` en ADA es más estricta que la de Pascal. Requiere que se manejen todos los valores posibles de la expresión del selector, ya sea explícitamente o con

when others .

```
declare
    puntos : Integer;
begin
    puntos := 3; -- Ejemplo de inicializacion

    case puntos is
        when 1 .. 5 =>
            Put_Line("No puede continuar");
        when 10 =>
            Put_Line("Trabajo terminado");
        when others =>
            Put_Line("Otro valor"); -- Manejar valores de 6..9
    end case;
end;
```

Posibles errores: En ADA, si el case no cubre todos los valores posibles y se omite la cláusula when others, se generará un error de compilación.

Sintaxis trasladada a C:

La sentencia switch de C, es equivalente al case de Pascal. Pero tiene una diferencia, con la estructura case, después de ejecutarse, el control "pasa a la siguiente línea" a menos que se utilice case con una declaración break.

```
#include <stdio.h>

int main() {
    int puntos = 4;

    switch (puntos) {
        case 1:
        case 2:
        case 3:
        case 4:
        case 5:
            printf("No puede continuar\n");
            break;
        case 10:
            printf("Trabajo terminado\n");
            break;
        default:
            printf("Otro valor\n");
            break;
    }
```

```
}

return 0;
}
```

En break es importantísimo para evitar que se ejecute la siguiente línea.

Posibles errores:

- Olvidar el break: Puede pasar que se ejecuten líneas de manera no intencionada, esto lo solucionamos poniendo la sentencia break para hacer que salga del case y no continúe ejecutando sentencias del case.
- No incluir un default: Puede generar comportamiento inesperado si la variable 'puntos' tiene un valor que no es manejado por ninguna sentencia del case.

Ejercicio 9: Qué diferencia existe entre el generador YIELD de Python y el return de una función. De un ejemplo donde sería útil utilizarlo.

- return finaliza una función y envía un valor de vuelta a quien la llama.
- yield También devuelve un valor, pero *pausa* la ejecución de la función. El estado de la función se guarda, por lo que puede reanudarse más tarde. Este tipo de función se denomina generador.

```
def count_up_to(max):
    n = 1
    while n <= max:
        yield n
        n += 1

counter = count_up_to(5)
for num in counter:
    print(num)
```

Utilidad: yield Eficiente en el uso de memoria para procesar secuencias grandes. En lugar de generar una lista enorme y devolverla, un generador genera elementos uno por uno según sea necesario.

Ejercicio 10: Describa brevemente la instrucción map en javascript y sus alternativas.

La instrucción map(), es un método en JavaScript que crea una nueva matriz aplicando una función proporcionada a cada elemento de la matriz original.

```
const numbers = [1, 2, 3, 4];
const squares = numbers.map(function(num) {
  return num * num;
});

console.log(squares); // Salida: [1, 4, 9, 16]
```

Alternativas:

- `for...of` bucle: más detallado pero le brinda control directo.
- `forEach()` :Ejecuta una función para cada elemento, pero no crea una nueva matriz.
- `reduce()` :Se puede utilizar para transformaciones más complejas, incluidas aquellas que `map()` puede hacer, pero su propósito principal es reducir una matriz a un solo valor.

Ejercicio 11: Determine si el lenguaje que utiliza frecuentemente implementa instrucciones para el manejo de espacio de nombres. Mencione brevemente qué significa este concepto y enuncie la forma en que su lenguaje lo implementa. Enuncie las características más importantes de este concepto en lenguajes como PHP o Python.

Espacio de nombres (namespace): Un espacio de nombres es una forma de organizar elementos de código (como clases, interfaces, etc) para evitar conflictos de nombres. Al tener muchas clases (en proyectos grandes o al usar librerías externas, es posible encontrar clases con el mismo nombre.)

Los espacios de nombres evitan estos conflictos de nombres, permitiendo usar el mismo nombre para diferentes elementos, siempre y cuando se encuentren en espacios de nombres diferentes.

Java utiliza paquetes (package) como mecanismo para espacio de nombres.

- Los paquetes se declaran utilizando la palabra clave `package` al comienzo de un archivo fuente de java.
por ejemplo, `package com.example.myproject;`
Esta declaración pone a la clase o interfaz definida en ese archivo en el paquete especificado.

Características en Java (en comparación con PHP o Python):

- **Declaración explícita:** En Java, se *debe* declarar explícitamente el paquete al que pertenece una clase. Esto difiere de Python, donde, si bien los módulos proporcionan espacios de nombres, no es *necesario* incluir cada script en una estructura similar a la de un paquete.
- **Mapeo del sistema de archivos:** Los paquetes Java tienen una estricta convención de mapeo a la estructura de directorios del sistema de archivos. Por ejemplo, la clase `com.example.myproject.MyClass` normalmente estaría en una estructura de directorios como `com/example/myproject/MyClass.java`. Esto es importante para la forma en que el compilador y el entorno de ejecución de Java localizan los archivos de clase.
- **import Declaración:** Para usar clases de otros paquetes, se usa la `import` declaración. Esto permite referirse a las clases por su nombre simple (p. ej., `MyClass` en lugar de `com.example.myproject.MyClass`) dentro del código. Java también cuenta con importaciones con comodines (p. ej., `import com.example.myproject.*;`) para importar todas las clases de un paquete.
- **Sin importaciones circulares:** el sistema de paquetes de Java, combinado con su mecanismo de carga de clases, generalmente evita las complejidades de las importaciones circulares que a veces pueden ocurrir en Python.
- **Control de acceso:** Los paquetes también influyen en el control de acceso en Java. Las clases declaradas con acceso predeterminado (privado al paquete) solo son visibles dentro del mismo paquete.